

Stunden- und Urlaubsberechnung – PraxisZeit

Stand: Juni 2026 · App-Version 1.8.2 Diese Doku beschreibt **exakt**, wie PraxisZeit Soll-, Ist-, Überstunden- und Urlaubswerte berechnet. Alle Formeln sind aus `backend/app/services/calculation_service.py` hergeleitet. Bei Code-Änderungen an der Berechnung **diese Datei mitpflegen**.

Verwandt: ARC42.md · BACKEND-ARCHITEKTUR.md → Berechnungsmodell.

0. Inhalt

1. Grundbegriffe
2. Pro-Mitarbeiter-Konfiguration
3. Tagessoll
4. Ist-Stunden (`net_hours`)
5. Monats-Soll
6. Abwesenheits-Typen-Matrix
7. Saldo & Überstundenkonto
8. Urlaubskonto
9. Jahresabschluss / Carryover
10. Sonderfälle
11. Worked Examples – Vollzeit
12. Worked Examples – Teilzeit

1. Grundbegriffe

Begriff	Bedeutung
Soll	Stunden, die der MA an einem Tag/Monat arbeiten <i>müsste</i> (vertraglich)
Ist	Tatsächlich erfasste Arbeitszeit (<code>net_hours</code>) + gutgeschriebene Abwesenheiten
Saldo / Überstunden	<code>Ist - Soll</code> (positiv = Mehrarbeit, negativ = Minusstunden)
Tagessoll	Soll-Stunden für einen einzelnen Arbeitstag
Tagesprinzip	Urlaub wird in Tagen gezählt (§3 BUrlG): 1 freier Arbeitstag = 1 Urlaubstag
Carryover	Jahresübertrag von Überstunden und Resturlaub ins Folgejahr

Goldene Regel (CLAUDE.md): Das Wochensoll wird **nie** direkt aus `user.weekly_hours` gelesen, sondern immer über `get_weekly_hours_for_date(db, user, datum)` — nur so werden rückwirkende Stundenänderungen (`working_hours_changes`) tagesgenau berücksichtigt.

2. Pro-Mitarbeiter-Konfiguration

Diese Felder am `User` steuern die gesamte Berechnung:

Feld	Bedeutung	Beispiel VZ	Beispiel TZ
<code>weekly_hours</code>	Wochensoll in Stunden	40	20
<code>work_days_per_week</code>	Arbeitstage/Woche (Divisor fürs Tagessoll!)	5	3
<code>use_daily_schedule</code>	individueller Tagesplan statt gleichmäßiger Verteilung	false	true
<code>hours_monday ... hours_friday</code>	Stunden je Wochentag (nur bei <code>use_daily_schedule</code>)	–	8/8/8/0/0
<code>vacation_days</code>	Jahres-Urlaubsanspruch in Tagen	30	18
<code>track_hours</code>	Soll/Ist-Zählung aktiv? (false = leitende Angestellte)	true	true
<code>first_work_day</code> / <code>last_work_day</code>	Eintritt/Austritt (Soll/Urlaub anteilig)	optional	optional
<code>scheduled_start_<wd></code> / <code>scheduled_end_<wd></code>	Arbeitszeit-Fenster je Wochentag (#201)	optional	optional

Historie: Ändert sich das Wochensoll mitten im Jahr (Teilzeit-Wechsel), wird ein `WorkingHoursChange` mit `effective_from` angelegt. `get_weekly_hours_for_date` liefert für jeden Tag den damals gültigen Wert — alte Monate bleiben korrekt, neue rechnen mit dem neuen Soll.

3. Tagessoll (daily target)

3.1 Gleichmäßige Verteilung (`use_daily_schedule = false`)

```
Tagessoll = weekly_hours / work_days_per_week
```

weekly_hours	work_days_per_week	Tagessoll
40	5	8,00 h
20	5	4,00 h
20	2	10,00 h
24	3	8,00 h

⚠ Der Divisor ist `work_days_per_week`, **nicht** fix 5. Ein 3-Tage-MA mit 24 h hat 8 h/Tag, nicht 4,8 h.

3.2 Individueller Tagesplan (`use_daily_schedule = true`)

`get_daily_target_for_date(user, datum)` liest die für diesen **Wochentag** konfigurierten Stunden (`hours_monday ... hours_friday`). Beispiel `8/8/8/0/0` :

Wochentag	Mo	Di	Mi	Do	Fr	Sa/So
Tagessoll	8	8	8	0	0	0

3.3 Allgemeine Regeln

- **Wochenende** (Sa/So): Tagessoll immer `0`.
- `track_hours = false` : Tagessoll immer `0` (keine Stundenzählung).
- **Sondertage 24./31.12.** (`half_day` / `free`): Tagessoll wird mit Faktor `0,5` bzw. `0` multipliziert (siehe §10.2).

4. Ist-Stunden (`net_hours`)

Pro Zeiteintrag (`TimeEntry`):

```
net_hours = max( 0 , (end_time - start_time) - break_minutes )
```

- Auf **2 Nachkommastellen** gerundet.
- **Floor bei 0**: kann nie negativ werden (offene Einträge ohne `end_time` → 0).
- **Pause** wird in Minuten geführt und abgezogen.

4.1 Arbeitszeit-Fenster (#201)

Ist ein Soll-Fenster gesetzt (`scheduled_start_<wd>` / `scheduled_end_<wd>`), kappt

`work_window_service.clamp()` die Stempelzeit auf `[Soll-Beginn - Puffer, Soll-Ende + Puffer]` (`work_window_grace_minutes` , Default 15). Die **Rohstempel** bleiben in `raw_start_time` / `raw_end_time` erhalten (§16 ArbZG); `net_hours` und alle Salden rechnen mit der **gekappten** Zeit.

Beispiel: Soll-Fenster Mo 08:00–16:00, Puffer 15 min, MA stempelt 07:30–17:10:

	Roh	gekappt (effektiv)
Start	07:30	07:45
Ende	17:10	16:15
Pause	30 min	30 min
net_hours	(informativ)	$(16:15 - 07:45) - 0:30 = 8,00 \text{ h}$

4.2 Gutgeschriebene Abwesenheiten

Zusätzlich zu den Zeiteinträgen zählen **Fortbildung (TRAINING)** und **Krankheit (SICK)** als geleistete Ist-Stunden (§3 EntgFG): ihre gebuchten `hours` werden zum Ist addiert.

```
Monats-Ist =  $\Sigma$  net_hours(Zeiteinträge im Beschäftigungsfenster)
            +  $\Sigma$  hours(TRAINING + SICK im Beschäftigungsfenster)
```

5. Monats-Soll

`get_monthly_target(db, user, jahr, monat)` iteriert **jeden Kalendertag** des Monats:

```
für jeden Tag d im Monat:
    überspringe, wenn Wochenende (Sa/So)
    überspringe, wenn d außerhalb [first_work_day, last_work_day]    (#193)
    überspringe, wenn d Feiertag (tenant-scoped)
    überspringe, wenn d ein Abwesenheitstag ist, der das Soll reduziert
    tagessoll = get_daily_target_for_date(user, d)
    tagessoll × Sondertag-Faktor (24./31.12.)                        (#146)
    Monats-Soll += tagessoll
```

Welche Abwesenheiten reduzieren das Soll? Alle **außer** `TRAINING`, `SICK`, `OVERTIME` (siehe Matrix §6). D. h. **VACATION, OTHER, PAID_LEAVE** reduzieren das Soll (der MA muss an diesen Tagen nicht arbeiten); TRAINING/SICK reduzieren es nicht (sie zählen stattdessen als Ist).

6. Abwesenheits-Typen-Matrix

Wie jeder Abwesenheitstyp Soll, Ist und Urlaubskonto beeinflusst:

Typ	reduziert Soll?	zählt als Ist?	bucht hours	belastet Urlaubsbudget?	Effekt aufs Konto
VACATION (Urlaub)	✓ ja	✗ nein	Tagessoll des Tages	✓ ja	saldo-neutral; zieht Urlaubstag
SICK (Krank)	✗ nein	✓ ja	Tagessoll des Tages	✗ nein	saldo-neutral (Soll bleibt, Ist gutgeschrieben)
TRAINING (Fortbildung)	✗ nein	✓ ja	Tagessoll des Tages	✗ nein	saldo-neutral (zählt als gearbeitet)
PAID_LEAVE (bezahlte Freistellung)	✓ ja	✗ nein	Tagessoll des Tages	✗ nein	saldo-neutral wie OTHER, aber kein Urlaubsverbrauch
OTHER (sonstige)	✓ ja	✗ nein	Tagessoll des Tages	✗ nein	saldo-neutral
OVERTIME (Überstundenausgleich)	✗ nein	✗ nein (Ist = 0)	explizite Stunden	✗ nein	Soll bleibt, Ist = 0 h → Überstundenkonto sinkt um die geplanten Stunden

OVERTIME-Sonderregel (CLAUDE.md): Beim Überstundenausgleich bleibt das **Soll bestehen** und das **Ist ist 0 h** für den Tag — dadurch reduziert sich das Überstundenkonto. Soll wird **nicht** reduziert.

Buchung der hours: Bei Voll-Tag-Typen wird hours = **Tagessoll des konkreten Tages** gebucht (nicht der 8-h-Client-Default). Nur OVERTIME behält die explizit eingegebenen Stunden. Halbtage (half_day): hours = 0,5 × Tagessoll.

7. Saldo & Überstundenkonto

7.1 Monatssaldo

```
Monatssaldo = Monats-Ist - Monats-Soll
```

7.2 Kumuliertes Überstundenkonto

`get_overtime_account(db, user, jahr, monat)` summiert die Monatssalden **kumulativ**:

- **Mit Carryover:** Startwert = `YearCarryover.overtime_hours` (neuester ≤ Jahr), Iteration ab Januar dieses Jahres → kein Doppelzählen.
- **Ohne Carryover:** Start ab dem ersten Zeiteintrag, Startwert 0.

```
Konto = Startwert + Σ (Monats-Ist - Monats-Soll) über alle Monate bis (jahr, monat)
```

7.3 Year-to-Date (JTD)

`get_ytd_summary` summiert Tagessoll und Ist vom **1. Januar bis heute** und addiert den Carryover des Jahres:

```
JTD-Überstunden = JTD-Ist - JTD-Soll + carryover_hours(Jahr)
```

8. Urlaubskonto

`get_vacation_account(db, user, jahr)` liefert Budget, Verbrauch und Rest — **in Tagen (maßgeblich)** und in Stunden (informativ).

8.1 Budget

```
budget_days = vacation_days (anteilig bei Eintritt/Austritt) + carryover_days  
budget_hours = budget_days * Tagessoll
```

- **Tagessoll fürs Budget** = `get_daily_target(user)` = `weekly_hours / work_days_per_week` (Durchschnitt, **nicht** datumsabhängig).
- **Anspruch (vacation_days)**: wird pro MA konfiguriert. Empfehlung/Standard nach Tagesprinzip:

```
vacation_days = 30 * work_days_per_week / 5      (Vollzeit 5 Tage → 30; 3 Tage → 18; 4  
Tage → 24)
```

- **Pro-rata bei Eintritt/Austritt im Jahr** (`first_work_day` / `last_work_day`):

```
budget_days = vacation_days * beschäftigte_Monate / 12
```

Der angebrochene Eintritts-/Austrittsmonat wird taggenau anteilig gerechnet (z. B. Eintritt 15.04. → Restmonate = $(12-4) + 16/30 = 8,53 \rightarrow 30 \times 8,53/12 \approx \mathbf{21,3 \text{ Tage}}$).

8.2 Verbrauch (Tagesprinzip)

Nur **VACATION** belastet das Budget (PAID_LEAVE bewusst nicht). Pro Urlaubs-Abwesenheit:

```
used_days += hours / Tagessoll_DIESES_Tages      (Volltag → 1,0 ; Halbtage → 0,5)  
used_hours += hours                               (nur informativ)
```

Dadurch kostet ein Urlaubstag immer **genau einen Tag seines eigenen Tagessolls** — ein 10-h-Montag kostet nicht mehr als ein 4-h-Mittwoch. Beide = 1,0 Urlaubstag.

Zusätzlich: Ein Sondertag (24./31.12.), der als `free` und `counts_as_vacation` konfiguriert ist, verbraucht ebenfalls **1 Urlaubstag** je MA (sofern im Beschäftigungsfenster und nicht schon als echter Urlaub gebucht).

8.3 Rest

```
remaining_days = budget_days - used_days      (maßgeblich, Tagesprinzip)
remaining_hours = budget_hours - used_hours    (informativ)
```

8.4 Budget-Check beim Antrag

Beim Anlegen eines Urlaubs/Antrags wird **tagebasiert** geprüft:

```
benötigte_Tage = Anzahl_buchbarer_Arbeitstage * (0,5 wenn Halbtage, sonst 1,0)
```

„Buchbare Arbeitstage“ = Werktage im Zeitraum mit Tagessoll > 0 (Feiertage/Wochenenden/ Null-Soll-Tage zählen nicht). Ein 3-Tage-Teilzeit-MA, der „eine ganze Woche“ Urlaub nimmt, verbraucht so nur **3** Urlaubstage.

9. Jahresabschluss / Carryover

`create_year_closing(db, jahr, users)` erzeugt für jedes Mitglied einen `YearCarryover` fürs Folgejahr (`jahr + 1`):

```
carryover.overtime_hours = get_overtime_account(user, jahr, 12)      # Saldo am 31.12.
carryover.vacation_days   = get_vacation_account(user, jahr).remaining_days # Resturlaub
```

Diese Werte gehen als Startwerte ins Folgejahr ein (Überstundenkonto-Start bzw. zusätzliche Budget-Tage). ⚠️ Offen (#191): Carryover für `track_hours = false` (leitende Angestellte).

10. Sonderfälle

10.1 Leitende Angestellte (`track_hours = false`)

- **Kein Soll/Ist:** Tagessoll = 0, keine Überstundenrechnung.
- **Urlaub trotzdem tagebasiert:** jeder VACATION-Tag = **1 Tag** (Halbtage nicht unterscheidbar → zählen als voller Tag), jeder `free + counts_as_vacation`-Sondertag = 1 Tag. Alle Stundenwerte bleiben 0; Budget folgt normaler Pro-rata- + Carryover-Logik.
- Response trägt `track_hours: false` (UI blendet Spalten aus).

10.2 Sondertage 24.12. / 31.12. (#146)

Tenant-Setting je Tag: `working_day` (Faktor 1), `half_day` (Faktor **0,5**) oder `free` (Faktor **0**). Bei `free` zusätzlich `counts_as_vacation` (zieht 1 Urlaubstag, §8.2). Der Faktor wird **nach** Wochenende/Feiertag/Abwesenheit angewandt, damit kein Tag doppelt behandelt wird.

10.3 Eintritt/Austritt (#193/#195)

Tage **vor** `first_work_day` oder **nach** `last_work_day` tragen weder Soll noch Ist (`_within_employment_window`). Das gilt symmetrisch an allen Per-Tag-Schleifen (Monats-Soll, Überstundenkonto, JTD und Ist-Seite) → keine Phantom-Überstunden durch Einträge außerhalb des Fensters.

⚠ **Ist `first_work_day` NICHT gesetzt** und existiert kein Carryover, beginnt `get_overtime_account` am 1. des Monats des **ersten Zeiteintrags** und `get_ytd_summary` am 1. Januar — das Soll der davorliegenden Tage wird mitgezählt → **Phantom-Minusstunden** bei unterjährigem Eintritt. Praxis-Empfehlung: bei jedem nicht-Januar-Eintritt `first_work_day` setzen (siehe HANDBUCH-ADMIN.md §4 Benutzerverwaltung).

11. Worked Examples – Vollzeit

Annahmen für alle Monatsbeispiele: Beispielmonat mit **22 Werktagen (Mo–Fr)**, davon **1 gesetzlicher Feiertag**. Werte gerundet auf 2 Nachkommastellen.

VZ-Profil

Feld	Wert
<code>weekly_hours</code>	40
<code>work_days_per_week</code>	5
<code>use_daily_schedule</code>	false
Tagessoll	40 / 5 = 8,00 h
<code>vacation_days</code>	30

Beispiel 11.1 – Monatssaldo mit Feiertag und 4 Urlaubstagen

```
Werktage im Monat           = 22
- Feiertage                  = 1
- Urlaubstage (VACATION)     = 4      (reduzieren Soll)
= Soll-relevante Arbeitstage = 17

Monats-Soll = 17 × 8,00 h = 136,00 h
```

Der MA arbeitet an den 17 Tagen, an 3 Tagen jeweils 0,5 h länger:

```
Monats-Ist   = 14 × 8,00 + 3 × 8,50 = 112,00 + 25,50 = 137,50 h
Monatssaldo  = 137,50 - 136,00      = +1,50 h   (Mehrarbeit)
```

Beispiel 11.2 – Krankheit statt Arbeit

Statt eines Arbeitstags ist der MA 1 Tag krank (SICK):

```
SICK reduziert das Soll NICHT → Soll bleibt 136,00 h (bei 4 Urlaub + 1 Krank: 22-1FT-4U = 17 Soll-Tage,
    der Kranktag ist einer dieser 17 → Soll zählt ihn voll)
SICK zählt als Ist           → 8,00 h gutgeschrieben (Tagessoll)
Saldo-Effekt des Kranktags    = 0 h   (8 h Soll, 8 h gutgeschrieben)
```

Beispiel 11.3 – Überstundenausgleich (OVERTIME)

MA nimmt 1 Tag Überstundenausgleich, gebucht mit 8,00 h:

```
Soll des Tages bleibt      = 8,00 h
Ist des Tages              = 0,00 h   (OVERTIME zählt nicht als Ist)
→ Überstundenkonto sinkt   um 8,00 h
```

Beispiel 11.4 – Urlaubskonto Vollzeit

```
Tagessoll (Budget) = 40 / 5           = 8,00 h
budget_days        = 30
budget_hours       = 30 × 8,00 h      = 240,00 h

Verbrauch: 4 volle Urlaubstage à 8,00 h
used_days   = 4 × (8,00 / 8,00)       = 4,0 Tage
used_hours  = 4 × 8,00                = 32,00 h

remaining_days = 30 - 4               = 26,0 Tage
remaining_hours = 240 - 32            = 208,00 h
```

12. Worked Examples – Teilzeit

TZ-A: Gleichmäßig 20 h / 5 Tage

Feld	Wert
<code>weekly_hours</code>	20
<code>work_days_per_week</code>	5
Tagessoll	$20 / 5 = 4,00 \text{ h}$
<code>vacation_days</code> (Empfehlung $30 \times 5/5$)	30

```
Monat (17 Soll-Tage wie 11.1):  
Monats-Soll = 17 × 4,00 h = 68,00 h  
  
Urlaub: 4 volle Tage à 4,00 h  
used_days = 4 × (4 / 4) = 4,0 Tage  
budget_days = 30 , budget_hours = 30 × 4 = 120 h  
remaining = 26,0 Tage / 104,00 h
```

Lehre: Auch ein 5-Tage-Teilzeiter hat **30 Urlaubstage** Anspruch — Teilzeit über die Stundenzahl/Tag, nicht über weniger Tage.

TZ-B: 24 h / 3 Tage (Mo–Mi, gleichmäßig)

Feld	Wert
<code>weekly_hours</code>	24
<code>work_days_per_week</code>	3
Tagessoll	$24 / 3 = 8,00 \text{ h}$
<code>vacation_days</code> ($30 \times 3/5$)	18

```
Tagessoll (Budget) = 24 / 3 = 8,00 h  
budget_days = 18  
budget_hours = 18 × 8 = 144,00 h  
  
„Ganze Woche" Urlaub: der MA arbeitet nur Mo/Di/Mi → nur 3 buchbare Arbeitstage  
benötigte_Tage = 3 × 1,0 = 3,0 Tage (Do/Fr haben Tagessoll 0 → zählen nicht!)  
used_days = 3 × (8 / 8) = 3,0 Tage  
remaining = 18 - 3 = 15,0 Tage
```

Lehre: Der Budget-Check ist **tagebasiert über buchbare Arbeitstage**. Eine Urlaubswoche kostet einen 3-Tage-MA nur 3 Tage, nicht 5.

TZ-C: Ungleichmäßiger Tagesplan (`use_daily_schedule = true`)

Tagesplan **Mo 10 h · Di 10 h · Mi 4 h · Do 0 · Fr 0** = 24 h auf 3 Tage, `work_days_per_week = 3`,
`vacation_days = 18`.

Wochentag	Mo	Di	Mi	Do	Fr
Tagessoll	10	10	4	0	0

Tagesprinzip beim Urlaub — jeder Urlaubstag kostet **1 Tag**, egal wie viele Stunden:

```
Urlaub Montag:    hours = 10,00 ; used_days += 10 / 10 = 1,0 Tag
Urlaub Mittwoch:  hours =  4,00 ; used_days +=  4 / 4  = 1,0 Tag
→ 2 Urlaubstage genommen = 2,0 Tage Verbrauch (NICHT 14h / Ø-Tagessoll!)
```

Budget (informativ in Stunden):
Tagessoll-Ø (Budget) = 24 / 3 = 8,00 h → budget_hours = 18 × 8 = 144 h
used_hours (informativ) = 10 + 4 = 14,00 h

Lehre: Tage sind maßgeblich, Stunden nur informativ. Bei ungleichmäßigem Tagesplan kann
`used_hours` von `Tage × Ø-Tagessoll` abweichen — das ist gewollt (Tagesprinzip §3 BUrlG).

TZ-D: Unterjähriger Eintritt (Pro-rata-Budget)

TZ-B-Profil (`vacation_days = 18`), Eintritt **15.04.**:

```
Tage im April          = 30
Resttage ab 15.04. (inkl) = 30 - 15 + 1 = 16
Restmonate             = (12 - 4) + 16/30 = 8 + 0,533 = 8,533
budget_days            = 18 × 8,533 / 12 ≈ 12,8 Tage (gerundet auf 0,1)
```

TZ-E: Rückwirkende Stundenänderung (Historie)

MA war bis 28.02. Vollzeit (40 h/5 = 8 h/Tag), ab 01.03. Teilzeit (20 h/5 = 4 h/Tag). Ein
`WorkingHoursChange(effective_from = 01.03., weekly_hours = 20)` wird angelegt.

```
Februar-Soll → 8 h/Tag (alter Wert, unverändert)
März-Soll    → 4 h/Tag (neuer Wert)
```

`get_weekly_hours_for_date` liefert pro Tag automatisch den damals gültigen Wert — kein manuelles Nachrechnen alter Monate nötig.

Quellen im Code

Funktion	Datei
<code>get_weekly_hours_for_date</code> , <code>get_daily_target(_for_date)</code>	<code>services/calculation_service.py</code>
<code>get_monthly_target / _actual / _balance</code>	<code>services/calculation_service.py</code>
<code>get_overtime_account</code> , <code>get_ytd_summary</code>	<code>services/calculation_service.py</code>
<code>get_vacation_account</code> , <code>create_year_closing</code>	<code>services/calculation_service.py</code>
<code>TimeEntry.net_hours</code>	<code>models/time_entry.py</code>
Sondertag-Faktoren	<code>services/special_days_service.py</code>
Arbeitszeit-Fenster	<code>services/work_window_service.py</code>
<code>hours</code> -Buchung (Tagessoll/Halbtage/OVERTIME)	<code>routers/absences.py</code> , <code>routers/admin_vacations.py</code>